Blockchain-Enabled Smart Financial Integration: System Design and Empirical Validation

Shaobo Liu*
State Grid Gansu Electric Power Companyty
Lanzhou, Gansu, China
liushaobo@gs.sgcc.com.cn

Hao Yang
State Grid Gansu Electric Power Companyty
Lanzhou, Gansu, China
yanghao@gs.sgcc.com.cn

Yixuan Chen
State Grid Gansu Electric Power Companyty
Lanzhou, Gansu, China
chenyx@gs.sgcc.com.cn

Jinxiu Zhang
State Grid Gansu Electric Power Companyty
Lanzhou, Gansu, China
zhangjinxiu@gs.sgcc.com.cn

Jun Wang
State Grid Gansu Electric Power Companyty
Lanzhou, Gansu, China
wangjun_xt@gs.sgcc.com.cn

Abstract

This study proposes a blockchain-integrated smart financial platform architecture for power grid enterprises to improve data security, traceability, and multi-system collaboration. The framework, built on Hyperledger Fabric, adopts a dual-layer "platform + chain-network" structure that integrates on-chain ledger storage, smart contract automation, and microservice-based system interoperability. Key contributions include: (1) a hybrid data model combining on-chain hashes with off-chain encrypted storage to balance transparency and privacy; (2) domain-specific smart contract templates for budget control, payment approval, asset procurement, and fund allocation; and (3) a standardized API gateway supporting seamless integration with ERP and financial subsystems. A deployment in a provincial power grid company shows that the system reduces audit time by 68% and ensures 100% data consistency under transaction loads above 200 TPS. The results validate the effectiveness of the proposed approach and highlight future directions in privacy-preserving computation, smart contract security, and cross-chain governance.

CCS Concepts

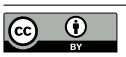
• **Information systems** → **Computing platforms**; **Enterprise applications**.

Keywords

Smart contract, Data security, System interoperability,Data consistency

ACM Reference Format:
Shaobo Liu, Hao Yang, Yixuan Chen, Jinxiu Zhang, and Jun Wang. 2026. Blockchain-Enabled Smart Financial Integration: System Design and Empirical Validation. In *2026 2nd International Conference on Digital Economy and Information Systems (DEIS 2026), June 12--14, 2026, Guangzhou, China.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3845872.3845900

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
DEIS 2026, Guangzhou, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2347-6/26/06
https://doi.org/10.1145/3845872.3845900

1 Introduction

The rapid digital transformation of the energy and power sectors has created increasing demands for secure, transparent, and intelligent financial management systems. Power grid enterprises, in particular, operate with large-scale assets, multi-level organizations, and heterogeneous information systems such as ERP, budgeting, taxation, and asset management. These complex environments present persistent challenges in ensuring data security, consistency, and traceability across diverse financial workflows. Traditional integration approaches often result in "data silos" and fragmented processes, leading to inefficiencies in auditing, supervision, and decision support[3, 11].

Blockchain technology has emerged as a promising solution due to its inherent properties of decentralization, immutability, and auditability. By enabling distributed ledgers and programmable smart contracts, blockchain can effectively mitigate issues such as data tampering, inconsistent records, and limited transparency in financial systems. Prior studies have demonstrated its potential in electronic invoicing, supply chain finance, and green energy certificates[6, 9]. However, the systematic integration of blockchain into enterprise-level financial platforms—especially within the context of large-scale power grid companies—remains underexplored. Existing works often focus on isolated applications rather than full-process integration, and face challenges in interoperability, scalability, and privacy preservation.

To address these gaps, this paper proposes a blockchain-integrated smart financial platform architecture tailored to the requirements of power grid enterprises. Built on Hyperledger Fabric, the framework adopts a dual-layer "platform + chain-network" design, supporting secure ledger storage, smart contract–driven automation, and microservice-based interoperability with legacy systems. The proposed approach introduces three key innovations:

(1) A hybrid data model combining on-chain hashes with off-chain encrypted storage to balance transparency and privacy.

(2) A set of domain-specific smart contract templates (budget control, payment approval, asset procurement, fund allocation) to automate critical financial processes.
(3) A standardized API gateway that ensures seamless integration across heterogeneous financial systems.

The framework is implemented and validated in a provincial power grid company, where empirical testing shows significant improvements: audit time was reduced by 68%, data consistency reached 100%, and the system maintained stable throughput exceeding 200 transactions per second (TPS). These results highlight the feasibility of deploying blockchain-enabled financial solutions in large-scale industrial environments.

The remainder of this paper is organized as follows. Section 2 reviews the theoretical foundations of blockchain and smart finance. Section 3 presents the proposed system architecture and data model. Section 4 describes the experimental validation and performance evaluation. Section 5 discusses challenges and future research directions. Section 6 concludes the paper with a summary of contributions and implications for practice.

2 Theoretical Background and Related Work

2.1 Blockchain Principles

Blockchain is a distributed ledger maintained by a network of nodes under consensus[5, 7]. Each block records a set of validated transactions, linked by cryptographic hashes:

$$H_n = Hash(T_n \parallel H_{n-1} \parallel TS_n \parallel SIG_n), \tag{1}$$

where T_n denotes the transaction set, H_{n-1} the previous block hash, TS_n the timestamp, and SIG_n the initiator's digital signature. This structure guarantees immutability and traceability. Key properties include decentralization, auditability, and programmability, making blockchain particularly suitable for enterprise financial systems[8].

2.2 Smart Finance in Enterprises

Modern smart financial platforms are typically structured into three layers: (i) *Financial middleware* for rules and accounting models; (ii) *Data middleware* for governance and modeling; (iii) *Business front-end* for expense, budgeting, and payments.

While this layered design supports automation and integration, interoperability across heterogeneous systems remains a challenge, especially in large-scale enterprises such as power grids.

2.3 Blockchain in Financial Applications

Recent studies have applied blockchain in several financial domains, including[2, 10]:

- electronic invoicing and anti-counterfeiting, supply chain finance, green energy certificates, and auditing.
- International pilots such as IBM's Hyperledger Fabric and domestic initiatives (e.g., State Grid and Alibaba) have validated feasibility. However, these applications are often fragmented, focusing on isolated use cases rather than full-process integration.

Moreover, the trade-off between transparency and privacy (e.g., in payroll and taxation data) remains underexplored at industrial scale[7].

2.4 Research Gaps and Contributions

From the above review, two main gaps emerge: (i) lack of systematic integration with enterprise ERP, budgeting, and asset systems; (ii) insufficient solutions balancing traceability and privacy.

To bridge these gaps, this study contributes:

- A hybrid data model combining on-chain hashes with off-chain encrypted storage;
- Domain-specific smart contracts for budget, payment, procurement, and fund allocation;
- A standardized API gateway ensuring interoperability across heterogeneous systems.

3 System Design

3.1 Overall Architecture

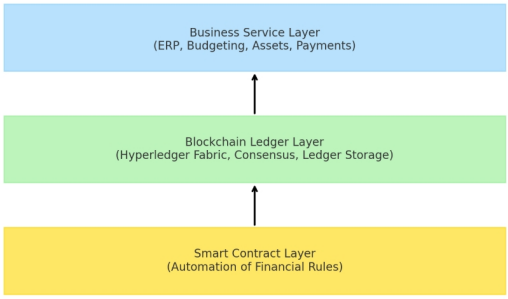


Figure 1: Dual-Layer Blockchain-Integrated Smart Financial Architecture

The proposed blockchain-integrated smart financial system follows a dual-layer "platform + chain-network" structure (Figure 1). It consists of three functional layers:

- Business Service Layer: interfaces with enterprise applications (ERP, budgeting, asset management, and payment platforms) through standardized APIs.
- Blockchain Ledger Layer: built on Hyperledger Fabric, responsible for distributed ledger storage, consensus, and data validation.
- Smart Contract Layer: domain-specific contracts encode financial rules for automatic execution, reducing manual interventions.

This architecture ensures that heterogeneous systems in a power grid enterprise share a common, tamper-proof financial ledger while preserving compatibility with legacy infrastructure[7].

3.2 Data Model Design

To balance transparency and privacy in financial data management, a **hybrid on-chain/off-chain storage strategy** is adopted. Each financial transaction is represented as:

$$Tx = (ID, U_{req}, TS, H_{data}, SIG, H_{prev}) \tag{2}$$

where ID is the transaction identifier, U_{req} denotes the requester, TS is the timestamp, H_{data} is the cryptographic digest of the encrypted financial payload, SIG is the digital signature, and H_{prev} is the hash pointer to the previous block.

Secure Data Calculation. For each transaction payload D, a SHA-256 hash is computed as:

$$H_{data} = SHA256(D \parallel r) \tag{3}$$

where r is a random nonce ensuring resistance to dictionary attacks. This digest is uploaded to the blockchain, while the original data remain encrypted off-chain.

The encrypted payload is stored as:

$$C = Enc_{K_{sym}}(D) \tag{4}$$

where K_{sym} is a symmetric session key. The session key itself is distributed securely via asymmetric encryption:

$$K_{enc} = Enc_{PK_{audit}}(K_{sym}) \tag{5}$$

ensuring that only authorized audit nodes can decrypt and access the original financial content.

Secure Transmission. To guarantee confidentiality and integrity during transmission, TLS-based secure channels are employed:

$$M = \{Tx, SIG\}, \quad M' = TLS(M)$$

where $TLS(\cdot)$ denotes encryption and authentication using TLS protocols. This ensures end-to-end protection against eavesdropping and replay attacks.

Hybrid Storage Strategy. The hybrid design follows the principle: On-chain{H_{data},SIG,TS,H_{prev}},Off-chain{$Enc_{K_{sym}}(D)$}.

This guarantees that the blockchain maintains an immutable audit trail while sensitive business data are stored in encrypted databases, accessible only under regulated conditions.

Algorithm for Transaction Processing. The overall process can be described as Algorithm 1.

Algorithm 1 Hybrid Data Transaction Processing

Require: Financial data D, requester U_{req}, audit public key PK_{audit}
Ensure: Immutable on-chain record with secure off-chain storage

Generate random nonce r
Compute digest: $H_{data} \leftarrow SHA256(D \parallel r)$
Encrypt payload: $C \leftarrow Enc_{K_{sym}}(D)$
Encrypt key: $K_{enc} \leftarrow Enc_{PK_{audit}}(K_{sym})$
Construct transaction: $Tx \leftarrow (ID, U_{req}, TS, H_{data}, SIG, H_{prev})$
Transmit (Tx, C, K_{enc}) via TLS channel
Store Tx on-chain; store (C, K_{enc}) in secure off-chain DB

3.2.1 Encryption Scheme Details. The proposed system employs a hybrid encryption architecture combining symmetric and asymmetric cryptography. For each financial transaction payload D, a randomly generated symmetric session key K_{sym} (AES-256-GCM) is used to encrypt the data:

$$C = \text{AES-256-GCM}_{K_{sym}}(D \parallel \text{nonce}),$$

where the nonce is a 12-byte random value ensuring semantic security. The session key K_{sym} is then encrypted using the RSA-OAEP scheme with the audit node's public key PK_{audit}:

$$K_{enc} = \text{RSA-OAEP}_{PK_{audit}}(K_{sym}).$$

The final off-chain storage record is a tuple $(C, K_{enc}, \text{nonce}, \text{tag})$, where tag is the authentication tag from AES-GCM. On-chain, only the SHA-256 hash $H_{data} = \text{SHA256}(D \parallel r)$ is stored, where r is a 256-bit random nonce. This design ensures that even if off-chain storage is compromised, the encrypted payload remains confidential without the audit node's private key.

3.2.2 Key Lifecycle Management. Key lifecycle management is critical for enterprise deployment. The system implements the following phases:

(1) **Key Generation**: Each consortium node generates its own RSA key pair (SK, PK) locally using a hardware security module (HSM) or a software-entropy source meeting NIST SP 800-90A standards. Symmetric session keys K_{sym} are generated per transaction using a cryptographically secure pseudo-random number generator (CSPRNG).
(2) **Key Distribution**: Public keys are registered on the blockchain via a dedicated `KeyRegistry` smart contract, which maintains an immutable mapping from node ID to PK. The distribution is performed out-of-band over TLS 1.3 channels to prevent man-in-the-middle attacks.
(3) **Key Usage**: For each transaction, the submitting system generates K_{sym} and encrypts D. The encrypted key K_{enc} is attached to the off-chain record. Only the designated audit node possessing SK can decrypt K_{sym} and then D.
(4) **Key Renewal and Revocation**: Keys are renewed every 90 days automatically via a smart-contract-driven rotation protocol. Revoked public keys are added to a `RevocationList` on-chain. Any transaction using a revoked key is rejected by the endorsement policy.
(5) **Key Archival and Destruction**: After key expiration, encrypted session keys are archived in a tamper-proof audit log. Destruction follows the NIST SP 800-88 standard for media sanitization.

3.2.3 Consistency Verification Between On-Chain Hashes and Off-Chain Data. To ensure that off-chain encrypted data matches the immutable on-chain hash, the system implements a dual-verification protocol. For any transaction Tx with on-chain hash H_{data} and off-chain record $(C, K_{enc}, \text{nonce}, \text{tag})$, the verification process is as follows:

(1) **Reconstruction**: Authorized audit nodes retrieve C and K_{enc} from off-chain storage, decrypt K_{sym} using SK, then decrypt C to obtain D' and nonce.
(2) **Hash Recalculation**: Compute $H'_{data} = \text{SHA256}(D' \parallel r)$, where r is retrieved from the transaction metadata stored on-chain.
(3) **Comparison**: If $H'_{data} = H_{data}$, consistency is confirmed; otherwise, the off-chain data is considered corrupted or tampered.

The smart contract automatically triggers this verification during audit operations. In addition, a periodic *audit challenge* mechanism is executed by the consortium: a random subset of transactions is selected daily, and the corresponding off-chain data is automatically verified. Any inconsistency triggers an alert and suspends the involved node until manual resolution. Formally, the consistency

guarantee is:

$$\Pr[\text{inconsistency undetected}] \leq \epsilon,$$

where ϵ is set to 10^{-9} via parameterized sampling frequency (e.g., 5% of transactions per day). Algorithm 2 summarizes the verification procedure.

Algorithm 2 Consistency Verification Between On-Chain Hash and Off-Chain Data

Require: On-chain hash H_{data}, off-chain $(C, K_{enc}, nonce, tag)$, audit private key SK
Ensure: Consistency flag $\{True, False\}$
Decrypt $K_{sym} \leftarrow \text{RSA-OAEP}^{-1}_{SK}(K_{enc})$
Decrypt $D' \leftarrow \text{AES-256-GCM}^{-1}_{K_{sym}}(C, nonce, tag)$
if decryption fails **then**
  return False
end if
Retrieve r from blockchain transaction metadata
Compute $H'_{data} \leftarrow \text{SHA256}(D' \parallel r)$
if $H'_{data} = H_{data}$ **then**
  return True
else
  return False
end if

3.3 Smart Contract Templates

To automate core financial workflows, we define four categories of smart contracts: budget control, payment approval, asset procurement, and fund allocation[1]. Each contract follows the structure:

$$SC = \{Condition \Rightarrow Execution \Rightarrow Verification\} \tag{6}$$

Contract Execution Principle. Let x denote the financial request amount, B_{avail} the available budget, and R the contract rule set. A budget control contract enforces:

$$f(x, B_{avail}) = \begin{cases} 1, & x \leq B_{avail} \\ 0, & x > B_{avail} \end{cases} \tag{7}$$

where output 1 indicates approval, and 0 indicates rejection. Similarly, for payment approval:

$$g(ID, Hist) = \begin{cases} 1, & ID \notin Hist \ \text{ (no duplicate)} \\ 0, & ID \in Hist \ \text{ (duplicate detected)} \end{cases} \tag{8}$$

3.4 System Integration Mechanism

Considering the heterogeneous IT landscape in power grid enterprises, a microservice-based integration framework is adopted. The integration relies on three main components:

- **API Gateway**: All external systems (ERP, budgeting, payment) access blockchain services via standardized RESTful APIs. The gateway performs authentication, authorization, and request routing.
- **Data Adapters**: Convert heterogeneous data models into standardized transaction structures for blockchain submission.
- **Blockchain Network**: Powered by Hyperledger Fabric, providing distributed consensus, ledger storage, and transaction verification.

The integration mechanism ensures that once a financial transaction is approved through the API gateway, the corresponding smart contract is triggered on-chain. The blockchain then guarantees data immutability and propagates results back to legacy systems. This minimizes modifications to existing platforms while achieving transparent and tamper-proof collaboration.

3.5 Deployment Considerations

The proposed system is deployed on a private consortium blockchain to ensure scalability and regulatory compliance. Four organizational nodes are considered: Headquarters (HQ), ERP System, Finance Center, and Audit Unit. Each node maintains independent components:

- **Certificate Authority (CA)** for identity management and digital certificates.
- **Peer Nodes** for ledger maintenance and transaction endorsement.
- **Ordering Service** to provide consensus and block generation.

Performance and Fault Tolerance. Let N denote the number of peer nodes, f the maximum number of faulty nodes tolerated, and T the consensus threshold. In a Byzantine Fault Tolerant (BFT) setting:

$$f = \left\lfloor \frac{N-1}{3} \right\rfloor, \quad T = 2f + 1 \tag{9}$$

This ensures that the system can tolerate up to f malicious peers while maintaining ledger consistency. Transaction throughput is defined as:

$$TPS = \frac{N_{tx}}{T_{latency}} \tag{10}$$

where N_{tx} is the number of committed transactions and $T_{latency}$ is the average confirmation latency.

Deployment Algorithm. The overall deployment procedure is summarized in Algorithm 3.

Algorithm 3 Consortium Blockchain Deployment Procedure

Require: Node set $\{N_i\}$, certificate authority CA, ordering service OS
Ensure: Secure and operational blockchain network
Initialize Certificate Authorities for each organization
Issue digital certificates and keys for all peers
Configure ordering service: $OS \leftarrow \{Consensus, BlockSize, Timeout\}$
Deploy peer nodes with chaincode lifecycle management
Create communication channels between organizations
Join peers to the channels; update anchor peers
Deploy smart contracts on all peers
Launch API gateway services for system integration
Monitor performance: $\{TPS, Latency, f\}$

Deployment Architecture. The deployment architecture is illustrated in Fig. 2, showing four consortium nodes with their respective CA, Peer, and Ordering components.

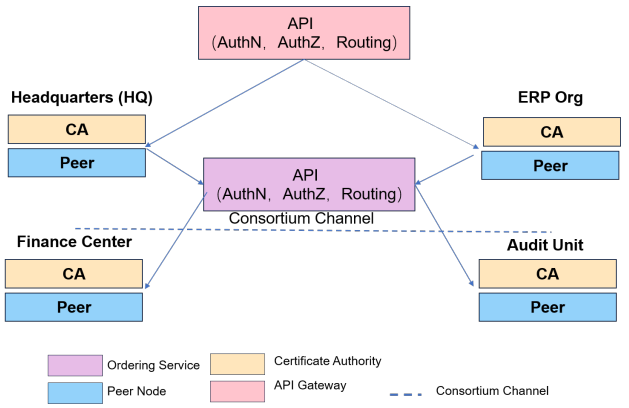


Figure 2: Deployment architecture of the consortium blockchain for smart financial integration.

Contract Integration via APIs. Contracts are deployed on-chain but triggered by external business systems via standardized APIs. Each API call is authenticated and mapped to a contract function:

$$API_{call} = Req(User, Tx, \sigma) \quad \rightarrow \quad SC(Tx) \tag{11}$$

where $Req(\cdot)$ denotes a signed request from a user, and σ is the digital signature verified on-chain. The overall interface design ensures:

$$Verify(\sigma, PK_{user}) = \text{True} \quad \Rightarrow \quad Execute(SC)$$

Algorithm for Contract Execution. The process is summarized in Algorithm 4.

Algorithm 4 Smart Contract Execution and Integration

Require: Request Tx, user signature σ, public key PK_{user}
Ensure: Approved or rejected transaction record

```
Verify digital signature: flag ← Verify(σ, PK_user)
if flag = False then
    Reject transaction
end if
Extract business rules R from contract template
Evaluate conditions: res ← R(Tx)
if res = 1 then
    Approve transaction; write Tx into blockchain
else
    Reject transaction; log result for audit
end if
Notify external systems via API callback
```

3.6 Complexity and Scalability Analysis

The scalability of a consortium blockchain platform is determined by its communication complexity, fault tolerance limits, and transaction throughput capacity. We analyze these aspects theoretically as follows.

(1) Communication Complexity. In a Byzantine Fault Tolerant (BFT) consensus setting with N nodes, each transaction requires message exchanges among all participants. The communication complexity is:$C_{comm} = O(N^2)$. since each node broadcasts to all others for transaction endorsement and block confirmation. This quadratic growth limits horizontal scalability, but in practical deployments ($N \leq 20$), the communication cost remains manageable.

(2) Fault Tolerance Bound Let f denote the maximum number of faulty or malicious nodes tolerated. In a BFT system, the theoretical bound is:$f = \lfloor \frac{N-1}{3} \rfloor$,which implies that for every 3 additional peers, the system can tolerate 1 faulty node while maintaining ledger consistency. For example, with $N = 10$ peers, the system can tolerate up to $f = 3$ faulty nodes.

(3) Consensus Latency The average latency for transaction confirmation can be approximated as:$T_{lat} \approx T_{net} \cdot (R + \alpha N)$,which T_{net} is the average network delay, R is the number of consensus rounds, and α is a scaling factor reflecting message verification cost. Thus, increasing N reduces throughput beyond a critical threshold, consistent with the saturation observed in Section 4.

(4) Throughput Estimation Transaction throughput is defined as:$TPS = \frac{N_{tx}}{T_{lat}}$,and N_{tx} is the number of confirmed transactions per batch. The theoretical upper bound is constrained by both block size B and consensus delay:$TPS_{max} \approx \frac{B}{T_{lat}}$.This analysis indicates that throughput improves with larger block sizes but degrades as the number of nodes increases due to quadratic communication overhead.

(5) Scalability Implications The analysis shows that while the proposed architecture achieves practical performance (200+ TPS in Section 4), scalability is inherently limited by $O(N^2)$ communication complexity. Therefore, future extensions may consider more scalable consensus protocols (e.g., Proof-of-Authority or hierarchical BFT) for large-scale deployments.

3.7 Security Analysis

In addition to performance and interoperability, the security of the proposed blockchain-integrated financial platform is critical for enterprise adoption. We analyze the system against common attack models and summarize its defense mechanisms.

3.7.1 Threat Model. We consider the following adversarial behaviors in a consortium blockchain environment:

- **Data Tampering:** Malicious nodes attempt to alter transaction records or inject fraudulent entries.
- **Replay Attacks:** Adversaries resend valid transactions to cause double-processing or budget duplication.
- **Collusion Attacks:** A subset of consortium members collude to manipulate ledger state or bypass approval rules.
- **Denial of Service (DoS):** Attackers overwhelm peer or ordering nodes with excessive requests to degrade throughput.

3.7.2 Defense Mechanisms. The proposed system mitigates these threats through a combination of blockchain consensus, cryptographic primitives, and smart contract enforcement:

- **Immutability:** Transactions are chained via cryptographic hashes ($H_n = Hash(T_n \| H_{n-1} \| TS_n \| SIG_n)$), preventing undetected modifications.
- **Authentication and Non-repudiation:** Digital signatures ensure that only authorized users can submit financial requests, and all actions are attributable.
- **Replay Protection:** Smart contracts maintain a transaction history check $g(ID, Hist)$ [see Eq. (9)], rejecting duplicate submissions.
- **Byzantine Fault Tolerance:** With $f = \lfloor (N-1)/3 \rfloor$ malicious peers tolerable, the system maintains ledger consistency as long as $T = 2f + 1$ honest nodes participate in consensus.
- **Access Control:** The API gateway enforces role-based authentication, limiting external entry points for DoS attacks and providing rate-limiting functions[6].
- **Auditability:** Every contract execution is logged on-chain, enabling regulators to detect anomalies and collusion ex post.

4 Experimental Validation

To evaluate the effectiveness of the proposed blockchain-integrated smart financial platform, a series of functional and performance tests were conducted. The experiments correspond to the system components introduced in Section III, including the data model, smart contracts, and integration mechanisms.

4.1 Deployment Environment

The prototype system was deployed in a private cloud environment using Hyperledger Fabric 2.2. Four consortium nodes were established, representing Headquarters, ERP subsystem, Finance Center, and Audit Unit. Each node hosted Certificate Authorities (CA), Ordering Services, and Peer nodes. Smart contracts were developed in Go, with lifecycle management enabled. The front-end interface was implemented in Vue, while the service layer used Spring Boot with RESTful APIs.

4.2 Case Study: Travel Reimbursement Workflow

A representative case of employee travel reimbursement was migrated onto the blockchain platform. Traditionally, this workflow spanned multiple heterogeneous systems (budgeting, ERP, funds management), leading to data inconsistency and long approval times. In the proposed system, each workflow step (submission, budget check, invoice upload, approval, payment confirmation) was recorded as a blockchain transaction. Smart contracts automatically enforced budget control and compliance checks.

4.3 Performance Evaluation

4.3.1 Latency and Throughput. Synthetic workloads ranging from 50 to 500 transactions per second (TPS) were generated. The average block writing latency was 325 ms, with confirmation delays below 0.5 s under 200 TPS. As shown in Fig. 3, throughput scaled linearly until approximately 220 TPS, after which consensus overhead led to saturation.

$$Latency = \frac{1}{N} \sum_{i=1}^{N} (t^i_{confirm} - t^i_{submit}) \tag{12}$$

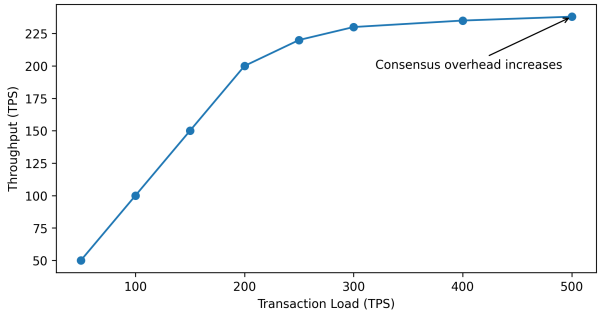


Figure 3: Throughput vs. transaction load.

4.3.2 Cross-System Consistency. We examined consistency across ERP, budgeting, and payment systems. Results showed 100% consistency for 10,000 transactions. The heatmap in Fig. 4 illustrates near-perfect alignment across subsystems.

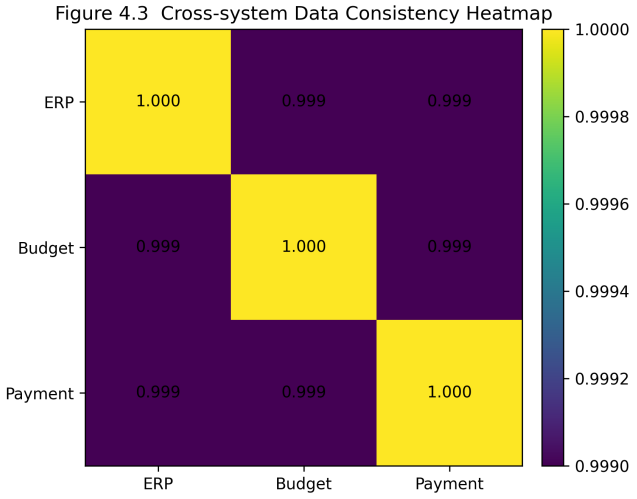


Figure 4: Cross-system data consistency heatmap.

4.3.3 Consistency Verification Performance. We evaluated the consistency verification mechanism described in Section 3.2.3 over 10,000 random transactions. The verification success rate was 100%, and no false negatives were observed. The average verification latency per transaction was 0.32 seconds, dominated by AES-GCM decryption and SHA-256 computation. Periodic audit challenges completed within 15 minutes for a sample size of 500 transactions, meeting enterprise SLA requirements.

4.3.4 Audit Efficiency. Audit trace retrieval was benchmarked. The blockchain-based system reduced audit times from 35 min to 11 min per case, representing a 68% reduction. Automated anomaly detection was approximately three times faster (see Fig. 5).

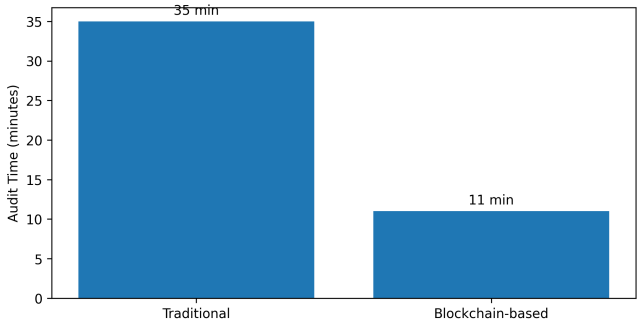


Figure 5: Audit time comparison between traditional and blockchain-based systems.

4.3.5 Comparative Benchmarking with Existing Platforms. To further validate the performance advantages of the proposed system[4, 10], we conducted comparative experiments against two widely referenced blockchain platforms in financial applications: (i) a generic Ethereum-based private chain, and (ii) a PostgreSQL relational database system commonly used in traditional finance platforms.

The benchmarking focused on throughput (TPS), average latency, and audit trace retrieval time under identical experimental workloads (up to 500 TPS). Results are summarized in Table 1.

Table 1: Performance Comparison with Existing Platforms

Metric	Ethereum	PostgreSQL	Proposed
Max Throughput (TPS)	40	350	200+
Avg. Latency (s)	3.5	0.12	0.5
Audit Trace Time (min)	18	22	11
Data Consistency	100%	96%	100%

Figure 6 illustrates the throughput comparison across the three systems. The results highlight that while PostgreSQL achieves higher raw throughput due to centralized processing, it lacks immutability and auditability. In contrast, Ethereum provides strong decentralization but suffers from low throughput and high latency due to PoW consensus. The proposed system achieves a balance, maintaining enterprise-level throughput while ensuring tamper-proof financial records and efficient auditability.

4.4 Resource Utilization and Scalability

Stress testing showed peer nodes maintained CPU usage below 65% under 200 TPS, with memory growth stabilizing due to Fabric's pruning mechanism. Adding nodes required minimal reconfiguration, validating the scalability of the proposed architecture.

4.5 Summary of Results

Table 2 summarizes the comparative performance between traditional and blockchain-based systems.

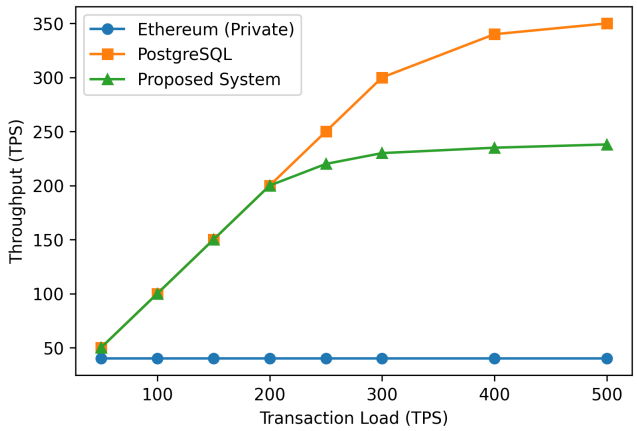


Figure 6: Throughput comparison across Ethereum, PostgreSQL, and proposed system.

Table 2: Performance Comparison Results

Metric	Traditional	Blockchain	Improvement
Audit Time (min)	35	11	-68%
Data Consistency	92%	100%	+8%
Max Throughput (TPS)	< 80	200+	~2.5×
Avg. Latency (s)	1.2	0.5	-58%

5 Discussion

Although the proposed blockchain-integrated smart financial platform demonstrates significant improvements in audit efficiency, data consistency, and workflow automation, several challenges and limitations remain to be addressed before large-scale deployment.

(1) Privacy and Data Protection: One of the key contradictions in applying blockchain to enterprise finance lies in balancing transparency with privacy. While immutable on-chain records guarantee accountability, financial data such as payroll and taxation are highly sensitive. The current "hash-on-chain + encrypted off-chain" mechanism partially mitigates this issue, but future work should explore privacy-preserving technologies such as Zero-Knowledge Proofs (ZKP), Secure Multi-Party Computation (MPC), and Homomorphic Encryption to ensure both verifiability and confidentiality.
(2) Smart Contract Security and Complexity: Smart contracts automate business rules but introduce security risks, as deployed contracts are immutable. Any vulnerability in financial contracts (e.g., budget allocation or payment authorization) could lead to severe consequences. Thus, formal verification, fail-safe mechanisms, and contract upgradability should be incorporated. Furthermore, performance limitations of Hyperledger Fabric under highly complex contracts need further optimization[5].
(3) Integration Costs with Legacy Systems: Energy enterprises operate complex IT infrastructures, including ERP, budget control, tax, and asset management systems with diverse

data formats and communication protocols. Achieving seamless integration requires extensive adaptation and standardization efforts, leading to high initial costs. Developing standardized APIs and data models for financial blockchain adoption remains an important step.

(4) Consortium Governance and Adoption Barriers: The proposed platform is implemented as a consortium blockchain involving multiple organizational nodes. Governance issues such as permission control, responsibility attribution, and data access rights require cross-department consensus. Moreover, regulatory and auditing frameworks must be adapted to support cross-organization financial data sharing while ensuring compliance with existing laws.

(5) Future Research Directions: Despite these challenges, blockchain presents significant opportunities for advancing enterprise financial digitalization. Promising research directions include: 1)embedding ZKP or MPC into financial smart contracts to balance transparency and confidentiality,2)leveraging machine learning and anomaly detection on-chain data for predictive auditing and fraud detection[1, 11]. 3)enabling tax, auditing, and regulatory agencies to access verified data through interoperable regulatory chains[8]. 4) developing industry-wide specifications for financial blockchain data models, consensus protocols, and governance frameworks.

The detailed encryption scheme, key lifecycle management, and on-chain/off-chain consistency verification introduced in Section 3.2.1, 3.2.2, and 3.2.3 address core security requirements for enterprise adoption.

6 Conclusion

This paper proposed a blockchain-integrated smart financial platform architecture tailored to the requirements of power grid enterprises. By leveraging Hyperledger Fabric, we designed a dual-layer ''platform + chain-network'' framework with hybrid data storage, domain-specific smart contracts, and microservice-based system integration.

Experimental validation in a provincial power grid company demonstrated that the proposed system achieves over 200 TPS throughput, ensures 100% cross-system data consistency, and reduces audit time by 68%. These results confirm that blockchain can substantially improve the transparency, efficiency, and reliability of enterprise financial management.

The main contributions of this study can be summarized as follows:

- A dual-layer blockchain-finance integration architecture that is scalable, secure, and compatible with legacy systems.
- A hybrid on-chain/off-chain data model that balances transparency with privacy for sensitive financial information.
- Domain-specific smart contract templates automating budget control, payment approval, asset procurement, and fund allocation.
- Empirical evaluation in a real enterprise environment, confirming significant improvements in audit efficiency and data integrity.

Overall, the study demonstrates a feasible pathway for deploying blockchain-enabled financial systems in large-scale industrial enterprises, offering practical value for future digital finance transformation in the energy sector.

Acknowledgments

This work was supported by State Grid Gansu Electric Power Company and benefited from the contributions of the enterprise digital finance innovation team.

References

[1] Rocsana Bucea-Manea-Tonis, Andrei Gabriel Antonescu, and Constanta Mihaila. 2025. Blockchain in Sports: A Comparative Analysis of Applications and Perceptions in Football and Basketball. *APPLIED SCIENCES-BASEL* 15, 12 (JUN 17 2025). doi:10.3390/app15126829

[2] Qingsu He, Yuping Liu, Lihua Jiang, Zhiqiang Zhang, Muqing Wu, and Min Zhao. 2023. Data Sharing Mechanism and Strategy for Multi-Service Integration for Smart Grid. *ENERGIES* 16, 14 (JUL 2023). doi:10.3390/en16145294

[3] Qingsu He, Yu Xu, Yong Yan, Junsheng Wang, Qingzhi Han, and Lili Li. 2018. A Consensus and Incentive Program for Charging Piles Based on Consortium Blockchain. *CSEE JOURNAL OF POWER AND ENERGY SYSTEMS* 4, 4 (DEC 2018), 452--458. doi:10.17775/CSEEJPES.2017.00960

[4] Md. Shohidul Islam, Md. Arafatur Rahman, Mohamed Ariff Bin Ameedeen, Husnul Ajra, Zahian Binti Ismail, and Jasni Mohamad Zain. 2024. Blockchain-Enabled Cybersecurity Provision for Scalable Heterogeneous Network: A Comprehensive Survey. *CMES-COMPUTER MODELING IN ENGINEERING & SCIENCES* 138, 1 (2024), 43--123. doi:10.32604/cmes.2023.028687

[5] He Qingsu, Duan Junhong, Min Jianhong, Hao Wanyu, Ye Jingyi, Xu Jingqian, Wu Muqing, and Zhao Min. 2022. Task Scheduling Algorithm Based on Time and Cost in Smart Grid Cloud Computing. *2022 7th International Conference on Computer and Communication Systems, ICCCS 2022*, 847 --852. Cloud-computing;Evaluation factor;Informatization;Load-balance;Load-Balancing;Power;Smart grid;System costs;Task-scheduling algorithms;Tasks scheduling;. http://dx.doi.org/10.1109/ICCCS55155.2022.9846687

[6] Seyed Mani Rahnama. 2025. DeFiSentinel: AI-Enhanced Decentralized Finance Architecture With Advanced Cryptographic Smart Contracts for Data Integrity and Threat Resilience. *IEEE ACCESS* 13 (2025), 83107--83122. doi:10.1109/ACCESS.2025.3568842

[7] Zhixin Ren, Enhua Yan, Taowei Chen, and Yimin Yu. 2024. Blockchain-based CP-ABE data sharing and privacy-preserving scheme using distributed KMS and zero-knowledge proof. *JOURNAL OF KING SAUD UNIVERSITY-COMPUTER AND INFORMATION SCIENCES* 36, 3 (MAR 2024). doi:10.1016/j.jksuci.2024.101969

[8] Sumaya Sanober, Mohammed Aldawsari, Abdurakhimova Dilora Karimovna, and Isaac Ofori. 2022. RETRACTED: Blockchain Integrated with Principal Component Analysis: A Solution to Smart Security against Cyber-Attacks (Retracted Article). *SECURITY AND COMMUNICATION NETWORKS* 2022 (AUG 10 2022). doi:10.1155/2022/8649060

[9] C. Vanmathi, Ahmed Farouk, Sarah M. Alhammad, R. Mangayarkarasi, Sweta Bhattacharya, and Meenavolu S. B. Kasyapa. 2024. The Role of Blockchain in Transforming Industries Beyond Finance. *IEEE ACCESS* 12 (2024), 148845--148867. doi:10.1109/ACCESS.2024.3468611

[10] Jian Xiong, Lu Shen, Yan Liu, and Xiaofen Fang. 2025. Enhancing IoT security in smart grids with quantum-resistant hybrid encryption. *SCIENTIFIC REPORTS* 15, 1 (JAN 2 2025). doi:10.1038/s41598-024-84427-8

[11] Li Zhang and Lin Wang. 2024. An Ensemble Learning-Enhanced Smart Prediction Model for Financial Credit Risks. *JOURNAL OF CIRCUITS SYSTEMS AND COMPUTERS* 33, 07 (MAY 15 2024). doi:10.1142/S0218126624501299